



Been-There — System Architecture

1. Purpose

Been-There helps people make better spending and going decisions by showing **which people they trust have already experienced a place** and how often they have been there.

The core question is:

| **Who do I know that has been here, and what was their experience?**

The product is intentionally different from traditional review platforms. Anonymous public ratings remain secondary; the primary signal is the user's trusted social graph and the real-world activity contributed by that graph.

2. Product Principles

- **People before ratings** — trusted relationships are more valuable than anonymous aggregate scores.
- **Experience before opinion** — repeat visits and activity history can be stronger signals than a one-time review.
- **Consent first** — users control whether their activity is visible and to whom.
- **Privacy by design** — collect the minimum information required to provide the product.
- **MVP before integrations** — prove the trusted-experience loop before building deep third-party integrations.
- **One modular application first** — avoid premature microservices.

3. MVP Scope

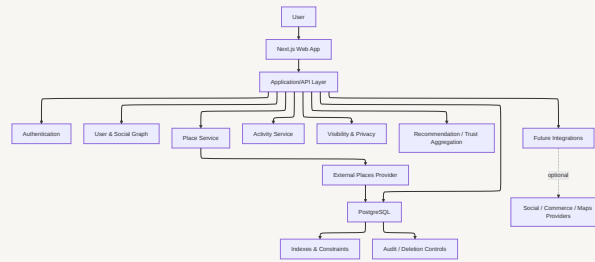
Core user journey

1. User creates an account.
2. User builds a trusted connection graph by connecting with people they know.
3. User searches for a place.
4. User records an activity at that place.
5. User can classify the experience: Recommend, Neutral, or Do not recommend.
6. User can record approximate/repeated visit count.
7. When another user opens the place, Been-There calculates which connected people have activity there.
8. The place page surfaces the number of connected people who have been there, recommendation distribution, repeat/regular visitors, and recent activity subject to visibility rules.

Explicitly out of MVP

- Instagram/Facebook/other social-history imports
- Swiggy/Zomato/food-order-history integrations
- Automated background location tracking
- ML recommendation models
- Microservices
- Kafka/event streaming infrastructure
- Complex recommendation ranking
- Mobile application

4. High-Level Architecture



Architectural decision

For the MVP, **Next.js + TypeScript is the application boundary for both UI and backend/API capabilities**. PostgreSQL is the source of truth.

This keeps deployment and development simple while preserving clear domain boundaries inside the codebase.

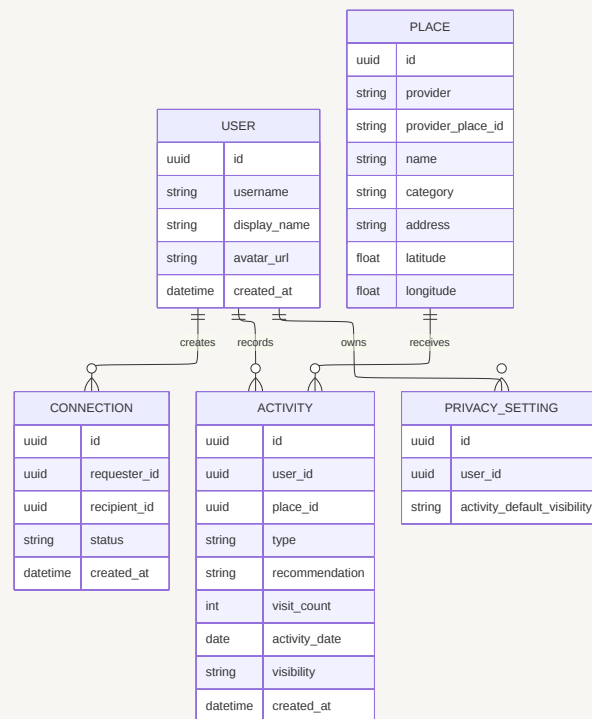
5. Proposed Technology Stack

Layer	Choice	Purpose
Web framework	Next.js	App Router, UI, server-side capabilities
Language	TypeScript	End-to-end type safety
UI	React	Product interface
Styling	Tailwind CSS	Fast consistent UI development
Components	shadcn/ui	Accessible reusable primitives
API	Next.js Route Handlers / server actions where appropriate	Application boundary
Validation	Zod	Runtime input validation
Database	PostgreSQL	Primary source of truth
ORM	Prisma or Drizzle	Typed database access
Auth	Auth.js / Supabase Auth / Clerk	Identity and session management

Layer	Choice	Purpose
Places	External Places API	Search and canonical place metadata
Deployment	Vercel	Web deployment
Version control	Git + GitHub	Source control

The ORM and authentication provider should be finalized immediately before implementation after evaluating migration ergonomics, database ownership, and privacy requirements.

6. Domain Model



7. Core Entities

User

Represents an authenticated Been-There account.

Important fields: internal immutable ID, display name, username, avatar reference, authentication identity mapping, and created timestamp.

Do not use external social-provider identifiers as Been-There's primary user ID.

Connection

Represents a relationship between two Been-There users.

Recommended lifecycle:

```
pending -> accepted
pending -> rejected
accepted -> blocked
accepted -> removed
```

The MVP can begin with a simple accepted/pending model, but the schema should leave room for blocking and removal.

Place

Represents a canonical real-world location.

A place should be internally identified by Been-There while retaining the provider and provider place ID used to discover/enrich it.

Example:

```
Been-There Place ID
BT_PLACE_123
+-- Provider
+-- Provider Place ID
+-- Name
+-- Branch
```

This prevents the application from making a third-party provider the permanent identity layer.

Activity

Represents a user's experience with a place.

MVP activity types: visited, ordered, with optional future types such as stayed, attended, or bought.

An activity is **not the same thing as a public review**. An activity may have no written text at all.

8. Trust Graph

The key query in Been-There is:

Given user U and place P, find the connected users of U who have visible activity for P.

Conceptually:

```
User A
+-- Friend B ---- Activity ---- Place P
+-- Friend C ---- Activity ---- Place P
+-- Friend D
```

The place page transforms this into a trusted evidence layer such as:

```
7 people you know have been here
5 recommend
1 neutral
1 do not recommend
3 are repeat visitors
```

9. Privacy Model

Privacy is a core domain concern, not a later feature.

Visibility levels

- private — only owner can see
- connections — accepted Been-There connections can see
- selected — future capability for selected users/groups

- public — optional future capability

For MVP, **private and connections** are sufficient.

Rules

1. A contact import or external integration must never make someone's activity visible without consent.
2. A user may hide or delete an activity.
3. Deleting an account must remove or anonymize its personally attributable activity according to the final retention policy.
4. Exact timestamps and precise location should not be exposed unless explicitly required and consented to.
5. The API must enforce visibility server-side; hiding an item in the UI is not a privacy control.
6. Analytics and aggregate counts should avoid revealing private activity through inference.

10. Trust Signal Model

The MVP should avoid machine-learning-based scoring.

Primary signals: number of connections who have been there, recommendation distribution, repeat visit count, and recency of activity.

Example:

```
Trust evidence
-----
7 connections have been here
23 combined visits
5 recommend
1 neutral
1 do not recommend
3 appear to be repeat visitors
```

A future **Trust Score** can be introduced only after enough usage data exists to validate the model.

For the MVP, show the underlying evidence rather than hiding it behind an opaque score.

11. API Surface

Initial application endpoints:

```
GET    /api/places/search
GET    /api/places/:id
GET    /api/places/:id/people
GET    /api/places/:id/activity-summary
POST   /api/activities
PATCH /api/activities/:id
DELETE /api/activities/:id
GET    /api/me/activities
POST   /api/connections
PATCH /api/connections/:id
DELETE /api/connections/:id
GET    /api/me/connections
GET    /api/me/profile
PATCH /api/me/profile
GET    /api/me/privacy
PATCH /api/me/privacy
```

The exact endpoint style can be revisited when the implementation approach is chosen, but the domain contracts should remain stable.

12. Critical Backend Query

The first high-value backend operation is:

```
getVisibleConnectedActivityForPlace(currentUserId, placeId)
```

Expected processing:

1. Resolve accepted connections for the current user.
2. Find activities for the requested place belonging to those users.

3. Apply each activity owner's visibility rules.
4. Aggregate safe summary metrics.
5. Return only fields allowed by the current viewer's relationship.
6. Sort individual activity evidence by a transparent rule such as recent activity or repeat frequency.

This operation should be covered by integration tests because it is the product's core value path.

13. Frontend Architecture

Recommended Next.js structure:

```
app/  
  (page routes)  
  api/  
    places/  
    activities/  
    connections/  
    me/  
  components/  
    domain/  
      places/  
      activities/  
      people/  
    ui/  
  lib/  
    auth/  
    db/  
    validation/  
    places/  
    privacy/  
    trust/  
    types/
```

Keep domain logic out of React components. A component should render data and trigger actions; privacy filtering, trust aggregation, and persistence belong in server-side/domain modules.

14. Main Screens

Home / Search

Primary action: search a place. The home experience should optimize for the question **Where are you thinking of going?**

Place page

Primary information hierarchy: place identity, **Your people have been here**, connection count, recommendation distribution, repeat visitors, recent visible activity, and general public information.

Activity creation

```
Place
↓
Have you been here?
↓
Visited / Ordered
↓
Recommend / Neutral / Do not recommend
↓
How often?
↓
Visibility
↓
Save
```

15. Data Integrity

Important database constraints:

- Unique username
- Unique provider + provider_place_id for canonical places
- Prevent duplicate active connection pairs
- Prevent invalid self-connections
- Validate activity ownership
- Enforce referential integrity with foreign keys

- Index connection lookup by requester/recipient
- Index activities by place_id and user_id
- Index activities by place_id + visibility

The database should enforce what the application cannot safely assume.

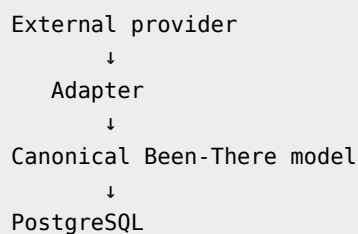
16. Security

Baseline requirements:

- Server-side authorization for every activity read/write
- Input validation with Zod
- Parameterized ORM queries
- CSRF-safe mutation strategy
- Rate limiting for high-abuse endpoints
- Secure session handling
- No client-provided user ID trusted for authorization
- Audit logging for security-sensitive actions
- Account deletion workflow
- Secret management via deployment environment configuration

17. External Integrations Strategy

Third-party integrations are adapters around the Been-There domain.



Never let provider-specific data structures leak throughout the application.

Future integrations may include places/maps providers, social identity providers, commerce/ordering platforms, and import sources.

An integration can enrich or import data only under explicit provider permissions and user consent.

18. Observability

MVP observability should include request/error logs, database error visibility, authentication failures, rate-limit events, integration failures, and basic product metrics.

Core product events:

```
place_search
place_view
activity_created
activity_updated
activity_deleted
connection_requested
connection_accepted
trusted_people_viewed
```

Avoid logging private activity details unnecessarily.

19. Deployment Architecture

Initial deployment:

```
GitHub
  ↓
Vercel
  ├── Next.js application
  └── Server-side API

PostgreSQL
  └── managed database

External Places Provider
```

There should be no need for a dedicated Kubernetes cluster or separate API server for the first version.

20. Development Phases

Phase 0 — Product + Architecture

- Freeze MVP boundaries
- Finalize data model
- Decide authentication strategy
- Decide PostgreSQL provider
- Document privacy rules
- Define API contracts

Phase 1 — Application Foundation

- Create Next.js + TypeScript project
- Configure linting / formatting
- Set up database and migrations
- Set up authentication
- Establish domain folder structure
- Add shared validation/types

Phase 2 — Social Graph

- Profiles
- Connection requests
- Accept/reject/remove
- Blocking groundwork
- Connection visibility rules

Phase 3 — Places + Activities

- Place search
- Canonical place records
- Create/edit/delete activity
- Recommendation signal
- Visit frequency
- Visibility

Phase 4 — Core Been-There Experience

- Place detail page
- Your people have been here
- Connected-person activity aggregation
- Repeat visitor signal
- Recency signal

Phase 5 — Validation

- End-to-end tests
- Privacy tests
- Abuse/rate-limit tests
- Performance checks
- Real-user feedback

Phase 6 — Integrations

Only after validating the core loop: identity providers, place enrichment, commerce/history integrations, and import mechanisms.

21. MVP Success Criteria

We should consider the MVP successful only if users repeatedly perform this loop:

```
Thinking about spending money
  ↓
Search place on Been-There
  ↓
See trusted people evidence
  ↓
Make / improve decision
  ↓
Later contribute own experience
```

The most important product metric is not total reviews. It is closer to:

Percentage of place views where a user finds useful trusted-person evidence.

Secondary metrics include connection acceptance rate, activity creation rate, repeat usage, place searches per active user, trusted-person evidence views, and return rate after first useful result.

22. Architecture Decisions to Revisit Later

- Prisma vs Drizzle
- Auth.js vs Supabase Auth vs Clerk
- Exact place provider
- Whether to support public activity
- Whether to add a formal trust score
- Whether activity should be event-based rather than count-based
- Contact discovery/import architecture
- Background jobs/queues
- Caching strategy
- Search engine beyond PostgreSQL/provider search

- Mobile client architecture

23. First Engineering Milestone

The first implementation target is:

A user can authenticate, connect with another user, record an activity for a place, and see that connection's visible activity when viewing the same place.

Everything else is downstream of this interaction.

```
Auth
  → Social Graph
  → Place
  → Activity
  → Privacy
  → Trusted Evidence
  → UI
```

24. Core Architecture Principle

Been-There is not a review database with a social feature.

It is a **trusted-experience graph** where places are the objects being evaluated and people are the primary source of evidence.

The architecture should preserve that distinction in the schema, domain services, APIs, and UI.